Generating Functions 6

Siddharth K

October 2026

Problem 2

We'll show that there is some driver that cannot park iff there exists i such that $|f^{-1}([i])| < i$ (there exists a parking spot i that fewer than i cars prefer). First, assume the latter. Then $> n - i$ have to park in the spots beyond i. There are $n - i$ of them, so you cannot park $> n - i$ people in them, so at least one cannot park.

For the other direction, assume that the car that wanted the ith spot cannot park, which means that all spots from i to n are taken. Say j is the highest available spot, so that $j < i$. Then $> n - j + 1$ people wanted some spot between $j + 1$ and n, so the number of people who want a spot between 1 and j is $\leq n - (n - j + 1) = j - 1 < j$, as desired.

Problem 3

Suppose we have n cars who want to park in $n+1$ spaces, but the cars can wrap around back to spot 1 if they don't find a spot between their preference and $n+1$. Then every car can park, and there will be one empty spot. Consider the arrangements where that empty spot is $n + 1$: every car's preferred spot is in $[n]$, and no cars wrap around past $n + 1$ (because then it would be occupied). Therefore all cars park in a way that satisfies the original parking problem. The number of such arrangements is

$$\frac{(n+1)^n}{n+1} = (n+1)^{n-1}.$$

Problem 4

Given a parking function f, we can form a tree on $\{0, 1, \ldots, n\}$ in this way: for each $i \in n$,

1. if $f(i) = 1$, connect i to 0,
2. otherwise, connect i to the car occupying the spot before car i's desired spot: $f(i) - 1$.

This is a tree because for each i, its parent is either 0 or occupies the spot $f(i)-1$, which is always less than the spot that car i ended up at. So following each parent follows a path of strictly decreasing parking spots until vertex 0. This means there are no cycles, so it is a tree.

We can reverse this process by defining f as

$$f(i) = (\text{parking spot that the parent of } i \text{ ended at}) + 1.$$

Problem 5

Given a rooted forest on the set $[n]$, we can construct one tree by adding a vertex labeled 0 and connecting it to the root of each tree in the forest. The resulting tree has $n+1$ vertices. To reverse this process, take a tree on the set $\{0, 1, \ldots, n\}$. Then remove 0, take the resulting components as the forest, and assign the root of each tree to be the vertex that was connected to 0 in the original tree. By Cayley's theorem, the number of trees on $\{0, 1, \ldots, n\}$ is $(n+1)^{n-1}$, so that is also the number of rooted forests on $[n]$.

Problem 6

Given a parking function on $[n]$, make an unrooted labeled tree on $n+1$ vertices using the bijection from problem 4, and then make a rooted forest from that tree using the bijection from problem 5 (and similarly in reverse).

Problem 8

We can define a labeled hierarchy recursively. Given a labeled hierarchy, there are two possibilities: either the tree consists of one labeled leaf, or the tree has an unlabeled root with at least 2 children. So the specification is

$$\mathcal{L} \cong \mathcal{Z} + \mathrm{SET}_{\geq 2}(\mathcal{L}).$$

So the generating function $L(z)$ satisfies

$$L(z) = z + e^{L(z)} - 1 - L(z),$$

or

$$L(z) = \frac{1}{2}(z + e^{L(z)} - 1).$$

I don't think it's possible to get an explicit formula for $L(z)$ given this.

Problem 9

This is the same as the last problem but the set must have size 2, so the specification is

$$\mathcal{M} \cong \mathcal{Z} + \mathrm{SET}_2(\mathcal{M}),$$

and the generating function satisfies

$$M(z) = z + \frac{1}{2}M(z)^2,$$

or

$$M(z)^2 - 2M(z) + 2z = 0.$$

Isolating $M(z)$ gives

$$M(z) = 1 \pm \sqrt{1-2z}.$$

Since $M(0) = 0$, we choose the negative solution:

$$M(z) = 1 - \sqrt{1-2z}.$$

By the generalized binomial theorem,

$$(1-2z)^{1/2} = \sum_{n=0}^{\infty} \binom{1/2}{n} (-2z)^n.$$

So for $n \geq 1$, the answer is

$$m_n = -n! \binom{1/2}{n} (-2)^n.$$

Problem 11

Given a function $f : [n] \to [n] \cup \{\perp\}$, consider its functional graph. Since some vertices map to $\perp$, it consists of several connected components where each component is either:

1. a rooted tree whose root maps to $\perp$, or
2. a directed cycle of rooted trees.

So the specification for $\mathcal{PF}$ is

$$\mathcal{PF} \cong \mathrm{SET}(\mathcal{T} + \mathrm{CYC}(\mathcal{T})),$$

where $\mathcal{T}$ is the class of labeled rooted trees. Therefore, the generating function is

$$PF(z) = e^{T(z)} \cdot e^{\log \frac{1}{1-T(z)}} = \frac{e^{T(z)}}{1-T(z)}.$$

Since

$$T(z) = ze^{T(z)},$$

we can differentiate both sides to get

$$\begin{aligned} T'(z) &= e^{T(z)} + ze^{T(z)}T'(z) \\ &= e^{T(z)} + T(z)T'(z), \\ T'(z)(1 - T(z)) &= e^{T(z)}, \\ T'(z) &= \frac{e^{T(z)}}{1 - T(z)}, \end{aligned}$$

which is the same as earlier, so

$$PF(z) = T'(z).$$

By Cayley's theorem, $t_n = n^{n-1}$, so

$$\begin{aligned} T(z) &= \sum_{n \geq 1} n^{n-1} \frac{z^n}{n!}, \\ PF(z) = T'(z) &= \sum_{n \geq 1} n^{n-1} \frac{z^{n-1}}{(n-1)!} \\ &= \sum_{n \geq 0} (n+1)^n \frac{z^n}{n!}. \end{aligned}$$

Comparing coefficients gives

$$pf_n = (n+1)^n,$$

which matches with the straightforward counting argument: $n+1$ output options for each of the n possible inputs.

Problem 13

Given a permutation with k minima, we can split the permutation before each minimum (except the first one), leaving k remaining blocks. Then use those blocks as cycles for a new permutation. To go in reverse, take the k cycles of the permutation, write the cycle so that the smallest element appears first, and then concatenate them in order of ascending first elements. The result is a permutation where each start of a cycle is now a minimum.